



Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Dispositivo Portátil para Automação de
Diagnóstico e Execução de *Scripts* em Ambiente Industrial

Relatório Final da unidade curricular Projeto

Afonso San Miguel, n.º 51792
João Fatelo, n.º 51815

Orientadores:

Professor Tiago Miguel Braga da Silva Dias
Professor Pedro Miguel Fernandes Sampaio

Julho de 2026

Agradecimentos

A concretização deste projeto final de curso não teria sido possível sem o apoio, a orientação e o incentivo de diversas pessoas, a quem gostaríamos de expressar a nossa profunda gratidão.

Em primeiro lugar, aos nossos orientadores, Professor Tiago Dias e Professor Pedro Sampaio, pela paciência, pelo profundo profissionalismo, pela generosa partilha de conhecimento e por toda a disponibilidade demonstrada ao longo deste percurso. A sua constante atenção ao detalhe, as sugestões, as críticas construtivas e o acompanhamento próximo foram fundamentais para o desenvolvimento e para a qualidade final deste trabalho.

Gostaríamos também de estender o nosso agradecimento ao Professor Rogério Rebelo e ao Professor Pedro Vieira. A sua análise e os contributos prestados na avaliação do relatório intermédio proporcionaram-nos uma perspetiva diferente sobre o nosso trabalho. As suas observações construtivas indicaram-nos caminhos valiosos de melhoria, que enriqueceram consideravelmente o resultado deste projeto.

Ao Instituto Superior de Engenharia de Lisboa, e em particular ao corpo docente da Licenciatura em Engenharia Informática, Redes e Telecomunicações, pela excelência da formação académica e pelas ferramentas que nos foram transmitidas ao longo destes anos.

Aos nossos colegas de curso, com quem partilhámos esta jornada, pelos momentos de debate, entreajuda e companheirismo que tornaram os desafios académicos muito mais fáceis de superar.

Às nossas famílias, em especial aos nossos pais, por todo o amor, sacrifício e apoio incondicional. A vossa confiança nas nossas capacidades e o vosso incentivo constante foram essências nesta jornada

A todos, o nosso muito obrigado.

Resumo

Nos ambientes industriais, o diagnóstico e a manutenção de servidores e equipamentos de rede dependem frequentemente da introdução manual de comandos através de interfaces de linha de comandos, exigindo conhecimento técnico especializado e limitando a autonomia de operadores. Este projeto teve como objetivo o desenvolvimento de um dispositivo portátil, baseado no microcomputador Raspberry Pi Zero WH, capaz de automatizar a seleção e execução de rotinas de diagnóstico previamente definidas e validadas, reduzindo a dependência de conhecimento técnico avançado e a probabilidade de erro operacional.

O dispositivo apresenta-se à máquina anfitriã como um periférico USB composto, combinando uma interface HID para a injeção automática de comandos, uma interface *Mass Storage* para o registo e disponibilização dos resultados obtidos, e uma interface ECM de apoio ao desenvolvimento. A seleção das rotinas é efetuada através de uma interface física local, composta por ecrã OLED e botões de navegação. O protótipo integra ainda mecanismos de verificação de integridade dos scripts por assinatura digital, deteção e adaptação automática à disposição do teclado da máquina anfitriã, e otimizações do processo de arranque do sistema operativo.

O trabalho desenvolvido abrangeu o enquadramento da solução face às alternativas comerciais existentes, a análise de requisitos, o estudo dos fundamentos USB relevantes, a definição da arquitetura do sistema e a implementação do protótipo, incluindo a plataforma *hardware*, o *firmware*, a interface HMI e os scripts de diagnóstico. Os testes realizados validaram o funcionamento integrado da cadeia principal do sistema, com recuperação bem-sucedida dos resultados em todas as execuções registadas e redução do tempo até à disponibilidade do menu de diagnóstico para cerca de 21 a 23 segundos, confirmando a viabilidade da solução proposta como alternativa de baixo custo às soluções comerciais analisadas.

Palavras-chave: Automação de diagnóstico; Raspberry Pi; USB composto; Assinatura digital; interface HM

Índice

1. Introdução	1
2. Enquadramento no Mercado	2
2.1. Hak5 USB Rubber Ducky.....	2
2.2. Hak5 Bash Bunny Mark II	3
2.3. Flipper Zero	4
2.4. Posicionamento da Solução Proposta	5
3. Análise de Requisitos	6
3.1. Requisitos Funcionais	6
3.2. Requisitos Não Funcionais	7
4. Fundamentos de USB para a Solução Desenvolvida	8
4.1. Sinalização Física e Codificação	8
4.2. Pacotes, Transações e Transferências	9
4.3. Endereçamento e Enumeração	9
4.4. <i>Endpoints</i> e Tipos de Transferência	10
4.5. Descritores e Classes de Dispositivo	10
5. Arquitetura Proposta	10
5.1. Visão geral do sistema	11
5.2. Fluxo de funcionamento	12
5.3. Justificação das principais decisões	13
6. Implementação do Protótipo	14
6.1. Plataforma hardware.....	14
6.2. Interface HMI	14
6.3. Configuração USB: HID, <i>Mass Storage</i> e ECM.....	15
6.4. <i>Firmware</i> da aplicação	16
6.5. <i>Scripts</i> de Diagnóstico	18
6.5.1. <i>Scripts</i> sem Privilégios de Administrador	18
6.5.2. <i>Scripts</i> com Privilégios de Administrador	19
6.6. Segurança da execução por assinatura digital	20
6.7. Compatibilidade de Teclado: Detecção e Seleção de <i>Layout</i>	23
6.8. Otimização do Tempo de Arranque.....	24
7. Testes e Validação	25
7.1. Ambiente de Teste	25

7.2. Metodologia de Validação	25
7.3. Resultados dos Testes Funcionais	26
7.4. Limitações da Validação	29
8. Conclusão e Trabalho Futuro	29
Referências	31

Lista de Tabelas

Tabela 1: Comparação das soluções existentes com a solução pretendida.....	5
Tabela 2: Mapeamento de pinos GPIO do protótipo	15
Tabela 3: Scripts de diagnóstico sem privilégios de administrador.....	18
Tabela 4: Scripts de diagnóstico com privilégios de administrador.....	19
Tabela 5: Ataques considerados e mitigação pelo mecanismo de assinatura digital.....	22
Tabela 6: Síntese dos testes funcionais realizados	27
Tabela 7: Evolução do tempo até à disponibilidade do menu de diagnóstico	28

Lista de Figuras

Figura 1 - USB Rubber Ducky	3
Figura 2 - Bash Bunny Mark II.....	3
Figura 3 - Flipper Zero	4
Figura 4 - Diagrama de blocos da arquitetura proposta	12
Figura 5 - Esquema de ligações elétricas do protótipo na breadboard.....	15
Figura 6 - Máquina de estados da aplicação HMI.....	17
Figura 7 - Diagrama de dependências dos módulos C e componentes do sistema	17
Figura 8 - Fluxo de assinatura fora da execução normal e verificação em tempo de execução dos scripts.....	22
Figura 9 - Medição final do tempo de arranque.....	28

Lista de acrónimos

CLI	<i>Command Line Interface</i>
CPU	<i>Central Processing Unit</i>
DHCP	<i>Dynamic Host Configuration Protocol</i>
ECM	<i>Ethernet Control Model</i>
GPIO	<i>General-Purpose Input/Output</i>
HID	<i>Human Interface Device</i>
HMI	<i>Human-Machine Interface</i>
I²C	<i>Inter-Integrated Circuit</i>
JSON	<i>JavaScript Object Notation</i>
NFC	<i>Near Field Communication</i>
NRZI	<i>Non-Return-to-Zero Inverted</i>
OLED	<i>Organic Light-Emitting Diode</i>
OSI	<i>Open Systems Interconnection</i>
OTG	<i>On-The-Go</i>
PID	<i>Product ID</i>
RAM	<i>Random Access Memory</i>
RFID	<i>Radio Frequency Identification</i>
RSA	<i>Rivest-Shamir-Adleman</i>
SCSI	<i>Small Computer System Interface</i>
SHA	<i>Secure Hash Algorithm</i>
SO	<i>Sistema Operativo</i>
SoC	<i>System on Chip</i>
SSH	<i>Secure Shell</i>
UDC	<i>USB Device Controller</i>
USB	<i>Universal Serial Bus</i>
VID	<i>Vendor ID</i>

1. Introdução

Nos ambientes industriais modernos, a manutenção e o diagnóstico de equipamentos, incluindo servidores e infraestruturas de rede, como *routers*, *switches* e *firewalls*, são tarefas críticas para assegurar a disponibilidade, a continuidade de serviço e a deteção atempada de falhas. Estas operações recorrem frequentemente a comandos específicos, executados através de interfaces de linha de comandos (CLI), e exigem conhecimento técnico sobre o sistema operativo, os serviços em execução e os parâmetros relevantes para diagnóstico. Esta dependência de procedimentos técnicos especializados constitui uma barreira à execução autónoma de tarefas de manutenção de primeira linha, dificultando a intervenção de operadores ou técnicos juniores e aumentando a dependência de elementos com formação mais avançada.

Para dar resposta a este problema, o presente projeto visou o desenvolvimento de um Dispositivo Portátil para Automação de Diagnóstico e Execução de *Scripts* em Ambiente Industrial, baseado num microcomputador Raspberry Pi Zero WH. Esta solução pretende permitir que procedimentos técnicos de diagnóstico, previamente definidos e validados, possam ser selecionados localmente e executados de forma automática na máquina anfitriã. Deste modo, operações que normalmente exigiriam a introdução manual de comandos numa interface de linha de comandos passam a poder ser realizadas através de uma interação simples e controlada, reduzindo a probabilidade de erro operacional e facilitando a execução de tarefas de manutenção de primeira linha por operadores ou técnicos com menor especialização.

O projeto resultou num protótipo funcional baseado no microcomputador Raspberry Pi Zero WH, que se liga à máquina anfitriã através de uma porta USB. O dispositivo apresenta-se ao sistema anfitrião como um periférico USB composto, permitindo automatizar a introdução de comandos e disponibilizar os resultados obtidos no próprio dispositivo. A seleção das rotinas de diagnóstico é realizada através de uma interface física simples, composta por um ecrã e botões de navegação.

O presente relatório descreve o trabalho desenvolvido ao longo do projeto final, desde a análise do problema e das soluções existentes até à conceção, implementação e validação do protótipo. No capítulo 2, é feito o enquadramento da solução no mercado atual, seguindo-se, no capítulo 3, a análise de requisitos que orientou o desenvolvimento. O capítulo 4 introduz os conceitos e tecnologias USB relevantes para a solução, sendo a arquitetura proposta apresentada no capítulo 5. A implementação do protótipo desenvolvido é detalhada no capítulo 6, incluindo a plataforma hardware, a interface HMI, a configuração USB, o firmware, os *scripts* de diagnóstico, os mecanismos de segurança, a compatibilidade de teclado e a otimização do arranque. O relatório termina com a apresentação dos testes realizados e a discussão dos resultados obtidos, no capítulo 7, e com as conclusões, limitações identificadas e propostas de trabalho futuro, no capítulo 8.

2. Enquadramento no Mercado

Em contexto industrial, entendido de forma ampla como o conjunto de infraestruturas técnicas que suportam a operação de uma organização, a manutenção e o diagnóstico podem incidir sobre servidores, equipamentos de rede, *firewalls*, *routers*, *switches* ou outros sistemas com interfaces de administração local ou remota. Nestas intervenções, é frequente o técnico ter de aceder ao equipamento a verificar, abrir uma sessão de terminal ou consola, executar comandos de diagnóstico ou *scripts* previamente disponíveis na máquina anfitriã, observar os resultados produzidos e recolher essa informação em ficheiros para posterior análise, registo ou encaminhamento.

Embora estas tarefas possam ser relativamente padronizadas, continuam muitas vezes dependentes da introdução manual de comandos ou da utilização de equipamento adicional, como um computador portátil ligado por rede TCP/IP, por consola série ou por outro meio de administração suportado pelo equipamento. Estas abordagens são adequadas em muitos cenários, mas podem introduzir complexidade operacional: exigem transporte de equipamento adicional, configuração da ligação, disponibilidade de credenciais, conhecimento dos comandos a executar e, em alguns casos, condições de rede ou permissões de acesso que podem não estar disponíveis no momento da intervenção.

Neste enquadramento, torna-se relevante considerar soluções portáteis capazes de automatizar parte do processo de diagnóstico através de uma ligação local simples e amplamente suportada. Entre as soluções existentes, destacam-se dispositivos USB que se apresentam à máquina anfitriã como periféricos de entrada, permitindo automatizar a introdução de comandos. A análise seguinte incide sobre os dispositivos comerciais mais próximos deste princípio de funcionamento, procurando identificar em que medida respondem ao cenário de utilização considerado neste projeto e que limitações permanecem por resolver.

2.1. Hak5 USB Rubber Ducky

O USB Rubber Ducky [9], lançado pela Hak5, é um dispositivo portátil concebido para automatizar a introdução de comandos numa máquina anfitriã. A sua função principal é executar sequências de teclas previamente programadas, permitindo abrir aplicações, escrever comandos e desencadear ações no sistema como se fossem introduzidas por um utilizador ao teclado. Para isso, o dispositivo apresenta-se ao sistema anfitrião como um teclado USB (HID) e executa instruções escritas em *DuckyScript*, uma linguagem própria para descrever sequências de teclas e respetivos tempos de espera.

Esta simplicidade torna o dispositivo eficaz para automatizar uma sequência específica de ações, mas limita a sua adequação ao cenário considerado neste projeto. Conforme se mostra na Figura 1, o USB Rubber Ducky apresenta-se externamente como uma simples pen USB e não disponibiliza ecrã nem botões que permitam selecionar, no momento da intervenção, uma entre várias rotinas de diagnóstico. Assim, a sequência a executar tem de estar previamente definida, o que reduz a flexibilidade em operações de manutenção onde o técnico pode necessitar de escolher diferentes diagnósticos consoante o equipamento ou a situação encontrada. Além disso, a ausência de uma interface própria para recolha e consulta imediata dos resultados torna-o menos adequado ao objetivo de automatizar diagnósticos industriais de primeira linha. O seu preço de cerca de 86 € é também elevado face à reduzida flexibilidade oferecida.



Figura 1 - USB Rubber Ducky

2.2. Hak5 Bash Bunny Mark II

O Bash Bunny Mark II [10], também da Hak5, é uma plataforma USB portátil baseada em Linux, concebida para automação de ações em máquinas anfitriãs e para operações de segurança ofensiva. Ao contrário do USB Rubber Ducky, não se limita à injeção de sequências de teclas: pode apresentar-se como diferentes dispositivos USB, incluindo teclado, armazenamento, adaptador de rede e porta série, isoladamente ou em combinação. Dispõe ainda de armazenamento interno, suporte para cartão MicroSD XC, Bluetooth Low Energy, interruptor físico de três posições para seleção da ação a executar e LED RGB para indicação do estado, conforme mostra a Figura 2. Segundo o fabricante, o dispositivo pode ficar operacional em cerca de sete segundos.

O Bash Bunny Mark II apresenta, assim, um conjunto de funcionalidades mais alargado do que o USB Rubber Ducky, sobretudo pela possibilidade de combinar diferentes interfaces USB e por executar sobre uma plataforma Linux. No entanto, no cenário de diagnóstico considerado neste projeto, continuam a existir limitações relevantes. A sua orientação principal é a realização de ataques USB multi-vetor e atividades de Red Team, embora também possa ser usado para automação de tarefas de TI. Além disso, a seleção por interruptor físico é adequada para um conjunto reduzido de ações, mas não oferece uma interface de menu que permita escolher facilmente, no momento da intervenção, entre uma biblioteca alargada de rotinas de diagnóstico. Por fim, o preço aproximado de 171 € e a necessidade de desenvolver as ações em DuckyScript e Bash podem constituir barreiras adicionais em contextos nos quais se pretende uma solução simples, económica e especificamente orientada para manutenção e diagnóstico.



Figura 2 - Bash Bunny Mark II

2.3. Flipper Zero

O Flipper Zero [11], representado na Figura 3, é um dispositivo multiferramenta portátil orientado para a exploração, análise e emulação de tecnologias usadas em sistemas de acesso e comunicação de curto alcance. Inclui funcionalidades associadas a RFID, NFC, sub-GHz, infravermelhos e iButton, dispondo ainda de ecrã monocromático de 128×64 pixéis e navegação física por botões. O fabricante apresenta-o como um dispositivo autónomo, que não exige computador ou *smartphone* para a utilização normal, embora utilize cartão microSD para armazenamento de dados e atualizações de *firmware*.

No contexto deste projeto, a funcionalidade mais relevante é o BadUSB, que permite ao Flipper Zero comportar-se como um dispositivo de entrada para automatizar ações na máquina anfitriã. No entanto, esta funcionalidade constitui apenas uma parte de um produto mais amplo, concebido como multiferramenta de segurança e não especificamente como equipamento de diagnóstico industrial. Além disso, a recolha dos resultados não é integrada no mesmo fluxo de execução da forma pretendida neste projeto: os dados gerados pela intervenção tendem a depender de armazenamento no próprio dispositivo ou de posterior consulta por outras vias, em vez de serem disponibilizados diretamente como resultado estruturado de uma rotina de diagnóstico.

Assim, apesar de a interface física do Flipper Zero (ecrã e navegação por botões) se aproximar do tipo de interação pretendido, o dispositivo não está orientado para a seleção e execução de uma biblioteca de diagnósticos com recolha imediata e padronizada dos resultados. O seu preço de aquisição, cerca de 239 €, é também elevado quando comparado com uma solução dedicada apenas à automação de diagnósticos.



Figura 3 - Flipper Zero

2.4. Posicionamento da Solução Proposta

A Tabela 1 sintetiza a comparação entre as soluções comerciais analisadas e a solução pretendida neste projeto. A comparação considera características relevantes para o cenário de utilização considerado, nomeadamente a capacidade de automatizar a introdução de comandos através de HID, a possibilidade de disponibilizar armazenamento à máquina anfitriã através de *Mass Storage*, a existência de uma interface local de seleção, o modo de seleção em tempo de execução, a plataforma tecnológica utilizada, o foco principal do produto e o custo aproximado. No caso do Flipper Zero e da solução pretendida, o valor indicado considera também a utilização de um cartão microSD de 32 GB; o USB Rubber Ducky inclui microSD de fábrica e o Bash Bunny Mark II dispõe de armazenamento interno.

Tabela 1: Comparação das soluções existentes com a solução pretendida

Característica	Rubber Ducky	Bash Bunny Mk II	Flipper Zero	Solução pretendida
Função HID (Teclado)	Sim	Sim	Sim	Sim
Função <i>Mass Storage</i>	Não	Sim	Não (BadUSB)	Sim
Interface local de seleção	Não	LED / Interruptor	Ecrã + D-Pad (128×64)	Ecrã + botões
Seleção em tempo de execução	Não, sequência pré-definida	Interruptor de 3 posições	Menu por D-pad	Menu navegável
Plataforma	Microcontrolador	Linux embebido	Microcontrolador	Dispositivo dedicado de baixo custo
Foco	<i>Automação HID / Pentesting</i>	Red Team / Automação USB	Multiferramenta de segurança	Diagnóstico industrial
Preço aproximado	~86 €	~171 €	~239 €	<35 €

Da análise comparativa resulta que as soluções existentes cobrem apenas parcialmente o cenário de diagnóstico considerado. O USB Rubber Ducky permite automatizar a introdução de comandos, mas não disponibiliza uma interface local de seleção nem um mecanismo integrado de disponibilização dos resultados. O Bash Bunny Mark II oferece maior flexibilidade técnica, incluindo a combinação de várias interfaces USB, mas a seleção por interruptor físico limita a escolha entre várias rotinas e o produto está orientado sobretudo para segurança ofensiva e automação técnica especializada. O Flipper Zero dispõe de uma interface física mais próxima da pretendida, mas a funcionalidade BadUSB é apenas uma componente secundária de uma multiferramenta mais ampla e não está orientada para a recolha estruturada de resultados de diagnóstico.

Assim, identifica-se uma oportunidade para desenvolver uma solução dedicada à manutenção e diagnóstico de primeira linha em contexto industrial, combinando automatização da introdução de comandos, seleção local de rotinas, disponibilização dos resultados no próprio dispositivo, operação simples para o utilizador e custo reduzido. Esta oportunidade fundamenta a proposta desenvolvida no projeto e será traduzida, na secção seguinte, na análise de requisitos da solução.

3. Análise de Requisitos

Com base no enquadramento do problema apresentado no capítulo 1 e na análise comparativa das soluções comerciais existentes discutida no capítulo 2, este capítulo sistematiza os requisitos que orientaram a conceção e o desenvolvimento da solução proposta. Os requisitos foram organizados em requisitos funcionais (subsecção 3.1), que definem as funcionalidades que o dispositivo deve disponibilizar, e requisitos não funcionais (subsecção 3.2), que estabelecem as condições de utilização, compatibilidade, segurança, custo e desempenho que a solução deve respeitar. Esta separação permite distinguir claramente entre o comportamento esperado do sistema e as restrições que condicionaram as principais opções de arquitetura, apresentada no capítulo 5, e de implementação, detalhadas nos capítulos seguintes.

3.1. Requisitos Funcionais

Os requisitos funcionais definem as funcionalidades que o dispositivo deve disponibilizar ao operador e à máquina anfitriã, decorrendo diretamente do cenário de utilização identificado no capítulo 1 e da lacuna de mercado caracterizada no capítulo 2.

- **RF1- Seleção local de rotinas de diagnóstico:** o dispositivo deve permitir ao operador selecionar, no próprio equipamento, a rotina de diagnóstico a executar, através de uma interface física simples. Este requisito decorre diretamente da limitação identificada nas soluções comerciais analisadas no capítulo 2, nas quais a seleção em tempo de execução é inexistente ou limitada a um número reduzido de opções fixas, e constitui o ponto de partida da interação descrita na arquitetura apresentada no capítulo 5.
- **RF2- Execução automática de comandos:** após a seleção da rotina, o dispositivo deve automatizar a introdução dos comandos necessários à sua execução na máquina anfitriã, dispensando o operador de escrever ou de conhecer os comandos específicos do sistema operativo em causa. Este requisito materializa o objetivo central do projeto, apresentado no capítulo 1, de reduzir a barreira técnica entre o operador e a interface de linha de comandos da máquina.
- **RF3- Suporte a múltiplas rotinas de diagnóstico:** a solução deve permitir disponibilizar várias rotinas de diagnóstico, organizadas de forma estruturada e selecionáveis, em vez de se limitar a uma única sequência de comandos pré-definida. Este requisito distingue a solução proposta de dispositivos como o USB Rubber Ducky (secção 2.1), cuja sequência de ações é fixa e definida previamente à intervenção.

- **RF4- Disponibilização dos resultados:** os resultados produzidos durante a execução dos diagnósticos devem ser guardados em ficheiros acessíveis a partir do próprio dispositivo, sem depender de infraestrutura de rede ou de acesso remoto à máquina anfitriã. Este requisito é satisfeito através da função *Mass Storage* integrada na arquitetura da solução, descrita no capítulo 5.
- **RF5- Indicação do estado da operação:** o dispositivo deve informar o operador sobre o estado da execução, distinguindo situações como execução em curso, conclusão com sucesso, erro de execução ou falha de integridade. Esta indicação, apresentada através da interface HMI, permite que o operador reconheça de forma imediata o resultado da intervenção, sem necessitar de consultar diretamente os ficheiros de resultado.
- **RF6- Verificação da integridade dos *scripts*:** antes de executar uma rotina, o dispositivo deve verificar se o respetivo *script* não foi alterado de forma não autorizada. Este requisito responde ao risco de o dispositivo poder ser usado indevidamente caso um *script* seja substituído por conteúdo malicioso, sendo satisfeito pelo mecanismo de assinatura digital detalhado no capítulo 6.6.
- **RF7- Suporte a diferentes disposições de teclado:** a solução deve permitir lidar com diferentes disposições de teclado configuradas na máquina anfitriã, de modo a reduzir erros na introdução automática dos comandos. Este requisito decorre da natureza da interface HID (secção 4.5), que transmite scancodes e não caracteres, sendo satisfeito pelo mecanismo de deteção e seleção da disposição de teclado apresentado no capítulo 6.7.

3.2. Requisitos Não Funcionais

Os requisitos não funcionais definem as condições de utilização, compatibilidade, segurança, custo e desempenho que a solução deve respeitar, complementando os requisitos funcionais ao estabelecer o enquadramento dentro do qual estes devem ser satisfeitos.

- **RNF1- Simplicidade de utilização:** a interação com o dispositivo deve ser simples e adequada a operadores ou técnicos com menor especialização, recorrendo a uma interface física clara e limitada às opções necessárias. Este requisito reflete o objetivo central identificado no capítulo 1: tornar acessível, a operadores sem formação avançada, a automação de diagnósticos técnicos.
- **RNF2- Compatibilidade com a máquina anfitriã:** a solução deve funcionar sem exigir a instalação de drivers específicos na máquina anfitriã, recorrendo a classes USB normalmente suportadas pelos sistemas operativos, conforme apresentado no capítulo 4. Este requisito assegura o funcionamento do dispositivo em modo plug-and-play, sem exigir configuração adicional por parte do operador.
- **RNF3- Portabilidade:** o dispositivo deve ser compacto e adequado a intervenções locais de manutenção e diagnóstico, podendo ser transportado e utilizado em diferentes localizações sem exigir equipamento adicional. Este requisito decorre diretamente do cenário de utilização em ambiente industrial descrito no capítulo 1.

- **RNF4- Alimentação pela ligação USB:** o dispositivo deve receber alimentação através da própria ligação USB usada para comunicação com a máquina anfitriã, sem necessitar de fonte de alimentação externa. Este requisito simplifica a utilização do dispositivo no terreno e reduz o número de ligações físicas necessárias à intervenção.
- **RNF5- Baixo custo:** a solução deve manter um custo reduzido face às alternativas comerciais analisadas no capítulo 2, de modo a ser viável como equipamento dedicado a tarefas de diagnóstico. Este requisito constitui um dos critérios de posicionamento da solução face aos dispositivos comerciais comparados na Tabela 1.
- **RNF6- Robustez operacional:** a solução deve reduzir erros associados à introdução manual de comandos e produzir resultados de diagnóstico de forma padronizada, independentemente do *script* executado. Este requisito é satisfeito pelo formato de saída comum a todos os *scripts* de diagnóstico, descrito no capítulo 7.3.
- **RNF7- Segurança da execução:** a solução deve mitigar o risco de execução de *scripts* adulterados, tendo em conta que a introdução automática de comandos pode ter impacto direto na máquina anfitriã. Este requisito é satisfeito pelo mecanismo de assinatura digital apresentado no capítulo 6.6, que impede a execução de *scripts* cujo conteúdo tenha sido alterado sem autorização.
- **RNF8- Tempo de arranque adequado:** o tempo entre a ligação do dispositivo e a disponibilidade do menu de diagnóstico deve ser reduzido, de modo a tornar a utilização prática em intervenções de manutenção. Este requisito é discutido em detalhe no capítulo 6.8, que descreve o processo de otimização do arranque do sistema.

4. Fundamentos de USB para a Solução Desenvolvida

A interface USB (*Universal Serial Bus*) [1] constitui um elemento central da solução desenvolvida, uma vez que permite estabelecer, através de uma única ligação física, a alimentação do dispositivo e a comunicação com a máquina anfitriã. No contexto deste projeto, o USB é usado para apresentar o dispositivo como um periférico composto, integrando funções de teclado, armazenamento e, em contexto de desenvolvimento, comunicação de rede. Este capítulo apresenta os conceitos de USB necessários para compreender estas opções, incluindo o funcionamento em modo dispositivo, o processo de enumeração, os *endpoints*, os descritores e as classes USB utilizadas na solução.

4.1. Sinalização Física e Codificação

O meio físico do USB é um cabo de quatro condutores: VBUS (5 V), GND, D+ e D-. A característica distintiva deste protocolo é a sinalização diferencial: os *bits* não são transmitidos como uma tensão absoluta, mas sim como a diferença de potencial entre D+ e D-. Esta técnica aumenta a imunidade ao ruído eletromagnético, uma vez que interferências comuns aos dois condutores tendem a ser rejeitadas pelo recetor. A codificação utilizada é NRZI (*Non-Return-to-Zero Inverted*): um '0' lógico é representado por uma transição no sinal, enquanto um '1' lógico corresponde à ausência de transição durante o intervalo de bit. Para manter transições suficientes para a sincronização do relógio no recetor, o protocolo

impõe o mecanismo de *bit-stuffing*, que consiste na inserção de um '0' após seis '1s' consecutivos. A detecção da velocidade de operação ocorre através de uma resistência de *pull-up* interna no pino D+ (*Full Speed*) ou D- (*Low Speed*).[1]

4.2. Pacotes, Transações e Transferências

Acima da camada física, o USB define três tipos de entidades: **Pacotes** (*token*, *data*, *handshake* ou *special*), **Transações** (sequências de pacotes que completam uma micro-operação, como *token* → *data* → *handshake*) e **Transferências** (abordadas mais à frente neste capítulo). O USB opera num modelo controlado pelo *host*: o *host* é o único que inicia a comunicação, e o dispositivo apenas responde quando endereçado.

No projeto, esta organização condiciona a forma como a solução comunica com a máquina anfitriã. O dispositivo não envia dados livremente para o *host*; em vez disso, apresenta-se como um periférico USB composto, com funções reconhecidas pelo sistema operativo. Assim, a automatização dos diagnósticos é feita recorrendo a classes USB padronizadas, nomeadamente teclado HID, armazenamento *Mass Storage* e, para apoio ao desenvolvimento, interface de rede ECM.

4.3. Endereçamento e Enumeração

O USB define um espaço de endereços lógicos de 7 *bits* (endereços 1 a 127). Quando o dispositivo é ligado, o *host* comunica com ele através do endereço universal *default* (0) e, durante o processo de enumeração, atribui-lhe um endereço exclusivo. A partir desse momento, cada *token* emitido pelo *host* inclui o endereço do destinatário, e todos os outros dispositivos ignoram os pacotes não endereçados a si. Esta simplicidade decorre da organização do barramento USB: não existem *routers* como nas redes IP; a expansão da topologia é feita por *hubs*, que não atribuem uma lógica própria de endereçamento aos dispositivos finais.

A Enumeração estabelece o diálogo inicial entre *host* e *device*: quando o cabo é ligado, o *host* deteta a presença do dispositivo pela alteração de tensão no pino D+, emite um *reset* de barramento, e inicia um diálogo estruturado com o *endpoint* 0 do dispositivo para ler os seus descritores. No final deste processo, o *host* identifica as interfaces disponibilizadas pelo dispositivo e carrega os *drivers* adequados e a sessão fica estabelecida. É este mecanismo que permite que um dispositivo USB composto seja reconhecido como um conjunto de funções distintas, por exemplo teclado, armazenamento ou interface de rede, desde que anuncie corretamente os respetivos descritores e classes USB.[1][4]

No contexto do projeto, este processo é relevante porque a solução depende de a máquina anfitriã reconhecer automaticamente as funções USB necessárias, sem configuração manual por parte do operador. A forma concreta como os descritores, identificadores VID/PID e funções USB foram configurados no protótipo é apresentada na secção de implementação da configuração USB.

4.4. Endpoints e Tipos de Transferência

Os canais lógicos de comunicação ponto-a-ponto do USB denominam-se *endpoints*. O *endpoint* 0 é usado para configuração e controlo; os restantes são definidos de acordo com as interfaces disponibilizadas pelo dispositivo. O protocolo especifica quatro tipos de transferência: *control* (configuração), *interrupt* (dados periódicos de baixa latência, como eventos de teclado), *bulk* (grandes volumes de dados sem garantia temporal, como transferências de ficheiros) e *isochronous* (dados com garantia de largura de banda, como áudio e vídeo). O tempo de barramento é dividido em *frames* de 1 ms: o *host* agenda transferências *interrupt* para dispositivos que exigem baixa latência, como teclados, e utiliza transferências *bulk* para dados sem requisitos temporais estritos, como os associados ao armazenamento. Desta forma, uma interface HID pode coexistir com uma interface *Mass Storage* no mesmo dispositivo composto, sendo a multiplexagem das transferências assegurada pelo protocolo USB e pelo sistema operativo do *host*.

No contexto deste projeto, esta distinção é importante porque a automatização de comandos depende de uma interface HID, adequada ao envio de eventos de teclado, enquanto a disponibilização dos resultados depende de transferências *bulk* associadas à função *Mass Storage*. A configuração concreta dos endpoints usados no protótipo é apresentada na secção de implementação da configuração USB.

4.5. Descritores e Classes de Dispositivo

Os descritores USB identificam o dispositivo e descrevem ao *host* as configurações, interfaces, *endpoints* e classes suportadas. A hierarquia organiza-se em quatro níveis: *device descriptor* (identificação global: VID, PID), *configuration descriptor* (requisitos de energia e número de interfaces), *interface descriptors* (declaração das funções, com os códigos de classe 0x03 para HID e 0x08 para *Mass Storage*) e *endpoint descriptors*. [1][4]

No contexto deste projeto, os descritores são importantes porque permitem que a máquina anfitriã reconheça o dispositivo como um periférico USB composto, com funções distintas. A interface HID exige ainda um *report descriptor*, uma estrutura binária padronizada que descreve o formato dos dados enviados, por exemplo modificadores e scancodes de teclado. É esta descrição que permite ao sistema anfitrião interpretar corretamente os eventos de teclado enviados pelo dispositivo.

5. Arquitetura Proposta

Este capítulo apresenta a arquitetura proposta para a solução desenvolvida, antes da descrição detalhada da sua implementação. O objetivo é explicar a organização geral do sistema, o papel dos seus principais componentes, o fluxo de funcionamento previsto e a forma como as rotinas de diagnóstico se integram na solução. Esta descrição permite separar a conceção da solução das opções concretas de implementação apresentadas nos capítulos seguintes.

5.1. Visão geral do sistema

A arquitetura proposta organiza-se em torno de três intervenientes principais: o operador, o dispositivo desenvolvido e a máquina anfitriã. O operador interage com o dispositivo através de um ecrã e um conjunto reduzido de botões. A interação entre os intervenientes pode ser analisada em dois planos distintos: o plano do operador e o plano da máquina anfitriã.

No plano do operador, o ecrã OLED e os três botões físicos constituem a interface Homem-Máquina (HMI). O operador navega no menu apresentado no ecrã e seleciona a rotina de diagnóstico pretendida. Esta seleção é processada pela aplicação/*firmware* do dispositivo, que mantém o estado da interface, gere a biblioteca de rotinas disponíveis e coordena a execução da rotina escolhida.

As rotinas de diagnóstico são organizadas como *scripts* externos à aplicação principal. Cada opção apresentada no menu da HMI está associada a um *script* da biblioteca de diagnósticos, o que permite separar a lógica de diagnóstico da lógica de controlo do dispositivo. Desta forma, novas rotinas podem ser adicionadas ou atualizadas sem alterar a aplicação/*firmware*, desde que respeitem a estrutura esperada.

Para permitir que a aplicação trate de forma uniforme os resultados produzidos por diferentes *scripts*, foi definida uma estrutura comum para as rotinas de diagnóstico. Cada *script* deve gerar um relatório composto por um cabeçalho de identificação, com informação como o nome do diagnóstico, a data e a máquina anfitriã, seguido de um corpo com os dados recolhidos. No final do relatório, a palavra **FIM** assinala o fim dos dados produzidos e a indicação **DIAG_EXIT** regista o código de saída da execução. Esta convenção permite ao *firmware* determinar automaticamente se o diagnóstico terminou com sucesso ou erro, sem ter de interpretar o conteúdo específico de cada relatório.

Antes de avançar para a execução da rotina selecionada, a aplicação/*firmware* verifica a assinatura digital do *script* associado. Esta verificação é colocada entre a seleção da rotina e a injeção de comandos, de modo a impedir que *scripts* adulterados sejam executados automaticamente.

No plano da máquina anfitriã, o dispositivo apresenta-se através de USB como um periférico composto, disponibilizando uma interface HID, que funciona como teclado virtual, e uma interface *Mass Storage*, que funciona como armazenamento em massa. Quando o operador seleciona uma rotina e a respetiva assinatura é validada, a aplicação/*firmware* injeta os comandos correspondentes através da interface HID. A máquina anfitriã recebe esses comandos como se fossem introduzidos por um operador ao teclado e executa-os localmente. Os resultados são guardados na área de armazenamento exposta pela interface *Mass Storage*, ficando disponíveis para leitura pelo próprio dispositivo.

A arquitetura inclui ainda uma interface adicional, *Ethernet Control Model* (ECM), que expõe uma ligação de rede virtual sobre o mesmo cabo USB e, esta interface não integra o fluxo funcional de diagnóstico, limitando-se a apoiar o acesso remoto ao dispositivo durante o desenvolvimento e a depuração do sistema.

A Figura 4 sintetiza esta organização sob a forma de um diagrama de blocos. O operador interage com a HMI, que comunica com a aplicação/*firmware* do dispositivo. Esta aplicação gere a seleção da rotina, a verificação da assinatura do *script* e a execução do fluxo de diagnóstico. A comunicação com a máquina anfitriã é feita através da interface HID,

para a introdução automática de comandos, e da interface *Mass Storage*, para a disponibilização e leitura dos resultados. A interface ECM surge separada do fluxo principal, sendo identificada apenas como apoio ao desenvolvimento e à depuração.

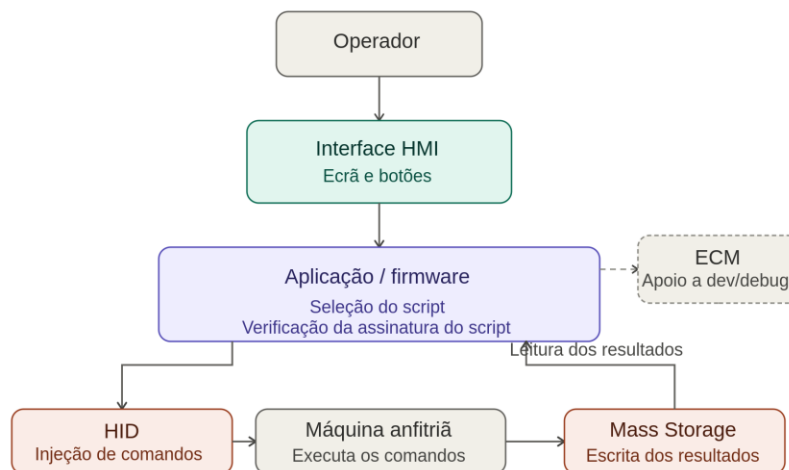


Figura 4 - Diagrama de blocos da arquitetura proposta

5.2. Fluxo de funcionamento

Esta subsecção descreve, de forma sequencial, o ciclo de vida de uma operação de diagnóstico, desde a ligação do dispositivo à máquina anfitriã até à disponibilização dos resultados ao operador.

- Ligação e reconhecimento:** o operador liga o dispositivo à porta USB da máquina anfitriã, que o reconhece como periférico USB composto, a aplicação/*firmware* arranca automaticamente com as funções necessárias ao fluxo de diagnóstico.
- Seleção da rotina:** o operador navega no menu da interface HMI e seleciona a rotina de diagnóstico pretendida, cada rotina corresponde a um *script* da biblioteca de diagnósticos do dispositivo.
- Verificação de integridade:** a aplicação verifica a assinatura digital do *script* associado, à rotina selecionada. Se a verificação falhar, a execução é interrompida sem que qualquer comando seja injetado na máquina anfitriã.
- Injeção de comandos:** confirmada a integridade, os comandos correspondentes são injetados na máquina anfitriã através da interface HID.
- Execução e escrita dos resultados:** a máquina anfitriã executa os comandos recebidos e escreve os resultados obtidos na área de armazenamento disponibilizada pela interface *Mass Storage* do dispositivo.
- Leitura e avaliação:** o dispositivo lê os resultados escritos pela máquina anfitriã e determina se a execução foi concluída com sucesso ou terminou em erro, com base na estrutura definida para os *scripts*, nomeadamente através da palavra **FIM** e da indicação final **DIAG_EXIT**.
- Apresentação do estado:** o dispositivo apresenta ao operador, através da HMI, o estado final da operação, disponibilizando o diagnóstico para consulta imediata.

Este fluxo repete-se de forma independente a cada nova seleção de rotina, sem necessidade de desligar ou reiniciar o dispositivo entre execuções.

5.3. Justificação das principais decisões

Esta subsecção justifica, de forma sucinta, as principais opções arquiteturais que sustentam a solução proposta.

- **Ligação USB única para alimentação e comunicação:** concentrar alimentação e comunicação numa única ligação USB simplifica a utilização do dispositivo em contexto de manutenção, reduzindo o número de cabos necessários à intervenção e satisfazendo o requisito RNF4.
- **Dispositivo USB composto:** a utilização de um dispositivo USB composto permite reunir, na mesma ligação física, as funções necessárias à solução. Esta decisão permite combinar a automatização da introdução de comandos, através da interface HID, com a disponibilização dos resultados, através da interface *Mass Storage*, mantendo a operação simples do ponto de vista da máquina anfitriã.
- **HID para automatização de comandos:** a classe HID permite que o dispositivo seja reconhecido pela generalidade dos sistemas operativos sem exigir *drivers* específicos, satisfazendo o requisito RNF2, e aproveita a confiança implícita atribuída pelo sistema operativo a um teclado para automatizar exatamente as ações que um operador executaria manualmente.
- **Mass Storage para disponibilização dos resultados:** expor uma área de armazenamento reconhecida nativamente pela máquina anfitriã evita depender de ligação de rede ou de *software* adicional para a recolha dos resultados, satisfazendo o requisito RF4 e reforçando a autonomia do dispositivo face à infraestrutura disponível no local da intervenção.
- **HMI simples para seleção local:** uma interface reduzida a um ecrã e a um pequeno número de botões, em vez de, por exemplo, um ecrã tátil, responde diretamente ao requisito de simplicidade RNF1, privilegiando a robustez física e a facilidade de operação por técnicos com menor especialização.
- **Estrutura comum dos *scripts* de diagnóstico:** a utilização de *scripts* externos permite separar a lógica de diagnóstico da aplicação/*firmware* do dispositivo. Para que esta separação não obrigue o *firmware* a conhecer os detalhes de cada diagnóstico, foi definida uma estrutura comum de saída, baseada num cabeçalho, num corpo de resultados, na palavra **FIM** e na indicação final **DIAG_EXIT**. Desta forma, novas rotinas podem ser adicionadas ou atualizadas sem alterar a lógica central da HMI e da comunicação USB, desde que respeitem esta estrutura.
- **Assinatura digital para verificação de integridade:** verificar a assinatura antes de qualquer injeção de comandos mitiga o risco de execução automática de *scripts* adulterados, respondendo ao requisito RNF7 e distinguindo a solução de ferramentas equivalentes sem garantias de integridade sobre o conteúdo executado.
- **ECM apenas como apoio ao desenvolvimento:** a interface ECM não decorre de nenhum requisito funcional de diagnóstico. A sua inclusão resulta exclusivamente da

conveniência de desenvolvimento e depuração, permitindo acesso remoto ao dispositivo sem interferir com o fluxo principal de utilização.

6. Implementação do Protótipo

Neste capítulo apresenta-se o protótipo desenvolvido, descrevendo os componentes de *hardware* selecionados e a forma como se interligam para concretizar a arquitetura proposta. A subsecção 6.1 detalha o microcomputador escolhido como elemento central do sistema, enquanto a subsecção 6.2 caracteriza a interface Homem-Máquina (HMI) e respetivas ligações elétricas. A configuração USB é descrita na subsecção 6.3, seguindo-se a implementação do *firmware* da aplicação na subsecção 6.4 e dos *scripts* de diagnóstico na subsecção 6.5. Por fim, as subsecções 6.6, 6.7 e 6.8 apresentam, respetivamente, o mecanismo de segurança por assinatura digital, o suporte a diferentes disposições de teclado e a otimização do tempo de arranque.

6.1. Plataforma hardware

O elemento central do protótipo é o Raspberry Pi Zero WH [7], um dispositivo de dimensões reduzidas (65 × 30 mm) baseado no SoC BCM2835 da Broadcom, com CPU ARM1176JZF-S a 1 GHz (*single-core*), 512 MB de RAM LPDDR2 e suporte completo a sistema operativo Linux. A versão WH inclui os 40 pinos GPIO já soldados (*header*), o que simplifica a ligação a componentes externos. O Raspberry Pi Zero WH possui duas portas micro-USB: uma dedicada exclusivamente à alimentação e outra com suporte a USB OTG (*On-The-Go*), que permite ao dispositivo operar quer como *host* quer como dispositivo USB. É esta segunda porta, ligada internamente ao controlador DWC2, que é utilizada para a ligação à máquina anfitriã. O sistema operativo reside num cartão microSD de 32 GB com Raspberry Pi OS Lite.[7]

A escolha desta plataforma justifica-se por reunir, num único dispositivo, as características necessárias à implementação do protótipo: suporte a Linux, capacidade de funcionamento em modo USB dispositivo, pinos GPIO para ligação da interface física, armazenamento em cartão microSD, dimensões reduzidas e preço compatível com o requisito de baixo custo definido para a solução. Estas características permitem concretizar a aplicação de controlo, a configuração do dispositivo USB, dispositivo composto e a ligação aos periféricos da HMI.

6.2. Interface HMI

Para interação com o utilizador, é utilizado um módulo OLED de 0,96 polegadas com controlador SSD1306, com uma resolução de 128×64 pixéis monocromáticos. A comunicação é feita via protocolo I²C (endereço 0x3C) utilizando apenas dois pinos do GPIO (SDA e SCL), por forma a reduzir a complexidade do circuito.[5]

A interface de navegação é assegurada por três botões táteis ligados diretamente aos pinos GPIO do microcomputador. Esta opção permite uma interação simples e adequada à seleção local das rotinas de diagnóstico, sem necessidade de teclado, rato ou monitor externos. O circuito encontra-se montado numa *breadboard*, o que facilitou a iteração sobre as ligações físicas e a validação do *hardware*. A Figura 5 ilustra a montagem elétrica entre o microcomputador, o ecrã OLED e os botões de controlo, enquanto a Tabela 2 apresenta o mapeamento dos pinos GPIO utilizados no sistema.

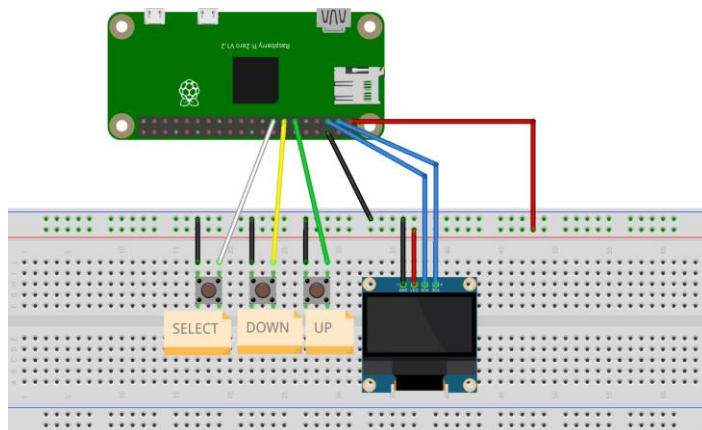


Figura 5 - Esquema de ligações elétricas do protótipo na breadboard

Tabela 2: Mapeamento de pinos GPIO do protótipo

Componente	Sinal	Ligação no Raspberry Pi	Observações
OLED SSD1306	SDA (I ² C)	GPIO 2 (SDA)	Barramento I ² C-1
OLED SSD1306	SCL (I ² C)	GPIO 3 (SCL)	Barramento I ² C-1
OLED SSD1306	VCC / GND	3.3V / GND	Alimentação
Botão UP	Sinal	GPIO 17	Pull-up interno ativo
Botão DOWN	Sinal	GPIO 27	Pull-up interno ativo
Botão SELECT	Sinal	GPIO 22	Pull-up interno ativo
Todos os botões	GND	GND	Ligação comum à terra

6.3. Configuração USB: HID, Mass Storage e ECM

A configuração USB é responsável por fazer com que o Raspberry Pi Zero WH seja reconhecido pela máquina anfitriã como um dispositivo USB composto. Esta configuração é essencial para o funcionamento do protótipo, uma vez que permite disponibilizar, através da mesma ligação física, as funções necessárias à execução dos diagnósticos e à disponibilização dos respetivos resultados [1].

Na implementação desenvolvida, o dispositivo composto integra três interfaces USB. A primeira é a interface HID, utilizada para emular um teclado virtual [2]. É através desta interface que o protótipo injeta na máquina anfitriã os comandos necessários à execução das rotinas de diagnóstico, como se estes fossem introduzidos por um utilizador através de um teclado físico.

A segunda interface é a *Mass Storage*, que permite apresentar à máquina anfitriã uma unidade de armazenamento [3]. Esta unidade é usada para disponibilizar os scripts e para guardar os ficheiros de resultados produzidos durante a execução dos diagnósticos. Do

ponto de vista da máquina anfitriã, esta interface é tratada como uma unidade de armazenamento USB convencional.

A terceira interface é a *Ethernet Control Model* (ECM), que cria uma ligação de rede virtual sobre USB. Esta interface não faz parte do fluxo normal de diagnóstico, sendo usada apenas durante o desenvolvimento e *debug* do protótipo. Através dela é possível aceder remotamente ao Raspberry Pi por SSH, sem necessidade de retirar o cartão microSD ou utilizar uma ligação de rede externa.

A criação do dispositivo composto é feita em Linux através de configs e do módulo *libcomposite* [4][16]. Em tempo de arranque, é executado um *script* que cria a estrutura do *gadget* USB, define os identificadores do dispositivo, configura as funções HID, *Mass Storage* e ECM, e associa essas funções à configuração ativa. A função HID inclui o respetivo *report descriptor*, que define o formato dos relatórios enviados à máquina anfitriã. A função *Mass Storage* é associada a um ficheiro-imagem, designado *piusb.bin*, que é apresentado ao *host* como unidade de armazenamento.

Um aspeto relevante desta implementação é a forma como o ficheiro-imagem usado pela interface *Mass Storage* é acedido pelo Raspberry Pi. Como o mesmo sistema de ficheiros não deve ser montado simultaneamente pela máquina anfitriã e pelo Raspberry Pi, o acesso local ao conteúdo de *piusb.bin* é feito de forma offline, através da biblioteca *mttools*. Esta biblioteca permite ler e escrever ficheiros diretamente numa imagem FAT, sem a montar no sistema operativo. Desta forma, após a execução de uma rotina de diagnóstico, o *firmware* consegue recuperar o ficheiro de resultados escrito pela máquina anfitriã, reduzindo o risco de corrupção da partição.[16]

O arranque da configuração USB é automático. Quando o protótipo é ligado à porta USB da máquina anfitriã, o Raspberry Pi recebe alimentação, inicia o sistema operativo a partir do cartão microSD e executa o *script* de configuração do *gadget* USB. Depois de ativado o dispositivo composto, a máquina anfitriã inicia o processo de enumeração e carrega os *drivers* correspondentes às classes USB configuradas. Assim, o protótipo consegue funcionar sem instalação manual de *drivers* específicos, recorrendo a classes USB normalmente suportadas pelos sistemas operativos.[1]

6.4. *Firmware* da aplicação

O *firmware* do dispositivo foi integralmente desenvolvido em linguagem C, sendo o seu comportamento global descrito por uma máquina de estados que dita o ciclo principal da aplicação, esta opção permite estruturar a interação com o operador de forma simples, separando os diferentes estados da interface e as ações associadas a cada evento.

O comportamento global da aplicação pode ser descrito sob a forma de uma máquina de estados, conforme ilustrado na Figura 6, após a inicialização bem-sucedida do ecrã e dos botões, o sistema entra no ciclo principal, onde aguarda continuamente eventos associados aos botões físicos. Consoante o estado atual (**STATE_MENU**, **STATE_RUNNING** ou **STATE_RESULT**) e o evento recebido, o sistema transita entre estados e atualiza a interface de forma correspondente. A captura dos sinais **SIGINT** e **SIGTERM** garante uma terminação limpa do processo.



```

graph TD
    main_c[main.c] --> button_c[button.c]
    main_c --> screen_c[screen.c]
    main_c --> menu_c[menu.c]
    button_c --> pigpio[pigpio  
(biblioteca GPIO)]
    screen_c --> i2c[/dev/i2c-1  
(Linux kernel)]
    menu_c --> scripts_c[scripts.c]
    scripts_c --> bash_scripts[Bash Scripts  
(sistema de ficheiros)]
  
```

■ Módulos do projeto (código C)
■ Dependências externas/sistema

Seguindo uma abordagem hierárquica de baixo para cima (das interações mais elementares com o hardware até à orquestração central), apresentam, de seguida, as funcionalidades associadas a cada módulo do programa:

- **Gestão de Entradas (button.c):** Na base da interação física, este módulo abstrai o *hardware* GPIO para ler o estado dos três botões. Implementa mecanismos de *debouncing* temporal por *software* (filtrando eventos num intervalo de 50 ms) e

deteção de transições de estado, garantindo que os ruídos mecânicos naturais dos botões não geram múltiplas entradas indesejadas.

- **Interface Gráfica (screen.c):** Também ao nível do *hardware*, funciona como um *driver* nativo para o ecrã OLED, comunicando diretamente através do barramento `/dev/i2c-1` do Linux. Gere o *framebuffer* em memória e agrega as funções de renderização, incluindo uma fonte de caracteres pré-definida (6×8) e a manipulação visual, como a inversão de cores (XOR bit-a-bit) para destacar visualmente a opção selecionada pelo utilizador.[5]
- **Execução de Tarefas (scripts.c):** Atuando ao nível do sistema operativo, este módulo mantém o registo dos diagnósticos disponíveis. Invoca a sua execução assegurando o redirecionamento dos resultados obtidos e dos *logs* de erro para os ficheiros apropriados.
- **Lógica de Navegação (menu.c):** Operando como a camada intermédia que interliga os módulos anteriores, este componente implementa a máquina de estados da interface (**STATE_MENU**, **STATE_RUNNING** e **STATE_RESULT**). É este módulo que consome os eventos capturados pelo gestor de entradas (button.c), orquestra o que deve ser desenhado pela interface gráfica (screen.c), incluindo o *scroll* do menu, e dá ordem de execução às tarefas (scripts.c) quando o utilizador seleciona um diagnóstico.
- **Módulo Principal (main.c):** No topo da hierarquia encontra-se o ponto de entrada do programa. Este ficheiro tem a responsabilidade de inicializar todos os módulos inferiores, manter o ciclo principal da aplicação em execução, e assegurar uma terminação limpa e segura, capturando sinais do sistema operativo como **SIGINT** e **SIGTERM**.

6.5. Scripts de Diagnóstico

Os *scripts* de diagnóstico constituem as rotinas executadas na máquina anfitriã após seleção pelo operador na HMI. No protótipo desenvolvido foram implementados sete *scripts*, organizados em duas categorias: cinco *scripts* sem privilégios de administrador e dois *scripts* que exigem privilégios de administrador.

6.5.1. Scripts sem Privilégios de Administrador

Os cinco de diagnóstico sem privilégios implementados no protótipo são: verificação de CPU, RAM, partições, processos e *uptime*. Estes *scripts* são invocados explicitamente através do *Bash* da máquina anfitriã, pelo que a presença do *Bash* no anfitrião é um requisito para a sua execução. A Tabela 3 apresenta os scripts desta categoria, indicando o ficheiro correspondente, o prefixo usado no log e os principais dados recolhidos.

Tabela 3: *Scripts de diagnóstico sem privilégios de administrador*

Script	Ficheiro	Prefixo do log	Dados recolhidos
CPU	cpu.sh	c	Modelo, nº de núcleos, utilização (%), <i>load average</i> , temperatura

RAM	ram.sh	r	RAM total/usada/disponível/livre, <i>swap</i> total/usada
Partições	particoes.sh	p	Ponto de montagem, total, usado (%), disponível, por partição real
Processos	processos.sh	ps	Total de processos; top 5 por CPU e por memória
<i>Uptime</i>	uptime.sh	u	Tempo desde o arranque, data de arranque, <i>load average</i>

Cada script foi ainda escrito de forma defensiva, testando a disponibilidade de cada comando ou ficheiro do */proc* antes de o usar e devolvendo o valor “indisponível” em vez de falhar quando uma métrica não pode ser obtida numa determinada máquina, por exemplo, a leitura de temperatura depende de */sys/class/thermal/thermal_zone0/temp*, que existe tipicamente em sistemas ARM (como o próprio Raspberry Pi) mas nem sempre no mesmo caminho em máquinas x86. O script *particoes.sh* lida com a sintaxe de saída do comando *df* filtrando apenas dispositivos de bloco reais (*/dev/**), para não listar sistemas de ficheiros virtuais irrelevantes para um diagnóstico industrial.

6.5.2. Scripts com Privilégios de Administrador

Um segundo grupo de diagnósticos exige privilégios de administrador na máquina anfitriã: a recolha de *logs* do sistema operativo e de ficheiros protegidos, tipicamente não acessíveis a um utilizador sem privilégios elevados. Estes dois *scripts* seguem a mesma organização geral dos scripts anteriores, mas exigem um mecanismo adicional de elevação controlada de privilégios. A Tabela 4 apresenta os *scripts* desta categoria.

Tabela 4: Scripts de diagnóstico com privilégios de administrador

Script	Ficheiro	Prefixo do log	Dados recolhidos
Logs do Sistema	logs.sh	l	Últimas entradas de <i>journalctl</i> / <i>dmesg</i> relevantes para diagnóstico de falhas recentes
Ficheiros Protegidos	protegidos.sh	pf	Permissões e integridade de ficheiros de configuração sensíveis (ex.: <i>sudoers</i> , <i>sshd_config</i>)

logs.sh recolhe as entradas mais recentes do registo do sistema operativo, por omissão, as últimas linhas de *journalctl* e de *dmesg*, filtrando por prioridade, nomeadamente avisos e erros, para que o relatório resultante se mantenha conciso e útil ao diagnóstico de uma falha recente, em vez de incluir a totalidade do *log* do sistema.

protegidos.sh verifica um conjunto pré-definido de ficheiros de configuração sensíveis da máquina anfitriã (por exemplo, */etc/sudoers*, */etc/ssh/sshd_config* e */etc/shadow*), reportando para cada um as permissões e o proprietário atuais e assinalando explicitamente qualquer permissão fora do esperado. Um ficheiro que devesse ser apenas legível pelo utilizador root e se encontre com permissões de escrita

mais abertas pode indicar uma configuração incorreta ou um possível comprometimento da máquina.

A execução desta categoria de scripts exige cuidados adicionais, uma vez que implica elevar privilégios na máquina anfitriã através da mesma interface HID usada nos restantes diagnósticos, introduzindo em tempo real a password de administrador do operador, gerida por um cofre de credenciais local ao dispositivo.

As credenciais são geridas através do script *credmgr.sh*, executado por SSH durante a configuração do dispositivo, que associa a cada máquina anfitriã conhecida, identificada por nome, a respetiva palavra-passe de administrador. Estes pares são cifrados com AES-256-CBC, com derivação de chave por PBKDF2, através do *openssl enc*, e guardados no ficheiro *credentials.enc*, protegido por uma chave gerada uma única vez (*.credkey*), com permissões restritas ao utilizador root. Tanto o ficheiro cifrado como a chave residem na raiz do sistema de ficheiros da Raspberry Pi, fora da partição *Mass Storage* exposta à máquina anfitriã, pelo que nunca são acessíveis a partir do lado do anfitrião.

Em tempo de execução, o módulo *creds.c* decifra o cofre através de um pipe direto para o *openssl*, mantendo o texto em claro apenas em memória, nunca em disco; é mantida em cache unicamente a lista de nomes de máquina, nunca as palavras-passe propriamente ditas. Antes de correr um *script* privilegiado, o operador seleciona no menu da HMI a máquina anfitriã correspondente, permitindo à função *creds_get_password()* obter a credencial associada a essa ligação.

A função *scripts_run_sudo()*, em *scripts.c*, encadeia esta credencial com as restantes garantias já descritas para os *scripts* sem privilégios: verifica primeiro a assinatura digital do script (secção 6.6) e só depois avança para a injeção do comando. Para reduzir o risco de a palavra-passe ser escrita com uma disposição de teclado incorreta, o comando de execução é construído numa única linha, que escreve, antes do próprio *sudo*, um marcador de verificação na partição do dispositivo, de forma semelhante ao mecanismo descrito na secção 6.7. Só depois de confirmar, pela leitura desse marcador, que a disposição configurada está correta, o *firmware* escreve a palavra-passe e confirma com Enter; caso o marcador não seja recuperado tal como esperado, a execução é interrompida antes de a palavra-passe chegar a ser escrita. Após a sua utilização, a palavra-passe é imediatamente apagada da memória do processo.

Este mecanismo permite ainda distinguir, no ecrã OLED, diferentes causas de falha específicas desta categoria de scripts, nomeadamente a inexistência de credencial guardada para a máquina selecionada, a disposição de teclado não confirmada e a palavra-passe incorreta, complementando a distinção entre **SUCESSO** e **ERRO** já garantida pelo formato **DIAG_EXIT** descrito na subsecção 6.5.1.

6.6. Segurança da execução por assinatura digital

O dispositivo desenvolvido partilha o princípio de funcionamento com ferramentas de injeção de teclado como o USB Rubber Ducky ou o Bash Bunny (ver o capítulo 2): qualquer conteúdo colocado na pasta de *scripts* é, por definição, executado com a confiança de um utilizador a escrever no teclado da máquina anfitriã. Em contexto industrial, onde o dispositivo pode ficar ligado sem supervisão contínua entre

diagnósticos, essa mesma propriedade que o torna útil constitui também o seu maior risco: a simples substituição de um ficheiro na área de armazenamento onde se encontram os *scripts*, por acesso físico não autorizado, bastaria para transformar uma ferramenta de diagnóstico numa ferramenta de ataque. O modelo de ameaça considerado centra-se, assim, na alteração ou substituição não autorizada dos *scripts* de diagnóstico e das respetivas assinaturas, assumindo que o atacante não tem controlo completo sobre o sistema do dispositivo, nem consegue alterar a aplicação responsável pela verificação ou a chave pública usada nesse processo. Este risco não constava nos requisitos formais do projeto, mas foi decidido mitigá-lo antes de considerar o sistema pronto para validação, através de um mecanismo de assinatura digital.

O mecanismo baseia-se em criptografia assimétrica RSA de 2048 *bits*, aplicada à assinatura de um resumo SHA-256 dos *scripts*. No protótipo desenvolvido, a chave privada é protegida por palavra-passe e mantida fora da área *Mass Storage* exposta à máquina anfitriã. Numa versão de produção, esta chave deveria idealmente ser mantida fora do dispositivo operacional, sendo usada apenas numa máquina de preparação e assinatura dos *scripts*. O *script* `prepare_pen.sh`, executado manualmente, sempre que a biblioteca de diagnósticos é atualizada, percorre todos os ficheiros `.sh` da partição, calcula o resumo criptográfico SHA-256 do nome do ficheiro concatenado com o seu conteúdo e assina esse resumo com a chave privada usando `openssl dgst[14]`, gravando o resultado numa pasta *signatures/* associada aos *scripts*.

Incluir o nome do ficheiro na assinatura, e não apenas o conteúdo, é uma decisão deliberada: sem esta medida, um atacante poderia copiar um *script* legítimo e assinado (por exemplo `cpu.sh`) e a respetiva assinatura para um nome diferente (por exemplo `particoes.sh`), fazendo a verificação aceitar conteúdo indevido sob outro nome. A inclusão do nome do ficheiro no resumo torna este reaproveitamento de assinatura ineficaz.

Em tempo de execução, antes de injetar qualquer comando via HID, a função `verify_signature()` em `scripts.c` repete o mesmo cálculo de resumo e verifica a assinatura correspondente com `openssl dgst -verify`, usando a chave pública residente localmente no sistema do dispositivo e fora da área *Mass Storage* exposta à máquina anfitriã. Esta última escolha, manter a chave pública de verificação fora da área acessível pela máquina anfitriã, fecha uma lacuna identificada durante o desenvolvimento: se a chave pública estivesse na mesma área onde se encontram os *scripts*, um atacante com acesso físico a essa área poderia gerar o seu próprio par de chaves, assinar um *script* adulterado com a chave privada falsa e substituir também a chave pública de verificação, fazendo a verificação passar com sucesso. Com a chave pública fixa fora dessa área, esse ataque deixa de ser possível dentro do modelo de ameaça considerado.

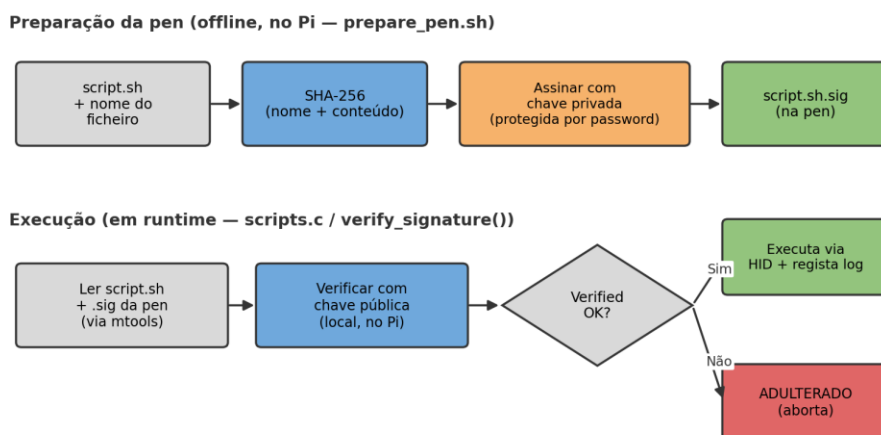


Figura 8 - Fluxo de assinatura fora da execução normal e verificação em tempo de execução dos *scripts*

Quando a verificação falha, porque o conteúdo foi alterado, a assinatura foi substituída ou não existe assinatura para o ficheiro, a execução é imediatamente abortada antes de qualquer tecla ser injetada na máquina anfitriã, e o ecrã mostra **ADULTERADO** em vez de **SUCESSO** ou **ERRO** (Figura 6), distinguindo claramente uma falha de integridade de uma falha de execução do próprio diagnóstico. Importa notar que este mecanismo garante a integridade e autenticidade dos *scripts* assinados, mas não garante, por si só, que o conteúdo de um *script* legitimamente assinado esteja correto ou seja seguro; essa garantia depende do processo de validação e aprovação dos próprios *scripts*.

Tabela 5: Ataques considerados e mitigação pelo mecanismo de assinatura digital

Cenário de ataque	Resultado com o mecanismo implementado
Alterar o conteúdo de um <i>script</i> na área acessível ao atacante	Resumo SHA-256 deixa de corresponder → ADULTERADO
Substituir a assinatura por outra existente	Nome do ficheiro não corresponde ao assinado → ADULTERADO
Gerar uma nova assinatura válida	Exige a chave privada, protegida por palavra-passe e não disponível na área <i>Mass Storage</i> exposta
Substituir a chave pública de verificação	Não é possível no modelo considerado: a chave pública não reside na área <i>Mass Storage</i> exposta
Acesso físico direto à Raspberry Pi	Risco residual, não mitigado por este mecanismo

O risco residual, um atacante com acesso físico direto e prolongado ao próprio dispositivo, não é mitigado por este mecanismo e é assumido como uma limitação, discutida no capítulo 8.

6.7. Compatibilidade de Teclado: Detecção e Seleção de *Layout*

A interface HID do dispositivo não transmite caracteres diretamente. Tal como um teclado USB, envia códigos de tecla (*scancodes*) que representam posições físicas do teclado, conforme definidos nas tabelas de uso USB para dispositivos HID [17]. É o sistema operativo da máquina anfitriã que traduz cada código de tecla para um carácter, de acordo com a disposição do teclado (*layout*) configurada nesse sistema. Um dispositivo HID que assuma sempre a disposição norte-americana (US), como é comum em ferramentas deste tipo, escreve caracteres errados sempre que a máquina anfitriã estiver configurada com outra disposição, particularmente nos símbolos usados no terminal, tais como barras, aspas, arrobas. Este comportamento poderia corromper silenciosamente os comandos injetados.

Para resolver este problema sem depender de introspeção do sistema operativo anfitrião, que o dispositivo, por design, não tem meios de consultar diretamente, a máquina de estado da aplicação HMI inclui o estado **STATE_LAYOUT** (ver Figura 6), que permite ao operador escolher manualmente a disposição de teclado da máquina anfitriã ou solicitar deteção automática.

O mapeamento de cada disposição de teclado está definido num ficheiro JSON próprio, localizado na pasta *languages/*, no mesmo formato de mapeamento modificador/*scancode* usado pelo projeto USB Rubber Ducky para as suas disposições de teclado. Esta convenção, já usada em ferramentas semelhantes, evita definir um novo formato de mapeamento. Os ficheiros JSON são interpretados pelo *firmware* através da biblioteca cJSON [18], adicionada às dependências do projeto e compilada como módulo próprio pelo *Makefile*. Estão atualmente disponíveis 21 disposições de teclado, incluindo us, gb, pt, es, fr, de, it, br, ch, be, cobrindo as configurações mais comuns em ambiente europeu e americano.

O operador pode seleccionar a disposição de teclado manualmente, escolhendo diretamente uma das opções apresentadas no ecrã. Neste caso, a escolha é assumida como correta e gravada em *config.txt*, sem qualquer teste adicional. Em alternativa, pode seleccionar a opção de deteção automática, apresentada como “Não sei”. Neste modo, o *firmware* testa as disposições candidatas por ordem de popularidade estimada. Para cada uma, escreve um marcador inócuo (**DIAGLAYOUTOK**) na própria partição de armazenamento através de um comando de teclado simples e verifica, lendo o ficheiro de volta pela mesma via *mttools* usada para os resultados dos diagnósticos, se o texto foi recebido intacto pela máquina anfitriã. A primeira disposição em que o marcador é recuperado sem alterações é assumida como a correta e gravada para as execuções seguintes.

Este teste não corre nenhum *script* nem afeta o sistema anfitrião: limita-se a escrever e ler um ficheiro dentro da própria partição do dispositivo. A disposição seleccionada, manual ou detetada, persiste entre execuções em *config.txt*, só sendo necessário repetir a escolha quando o dispositivo é ligado a uma máquina anfitriã diferente.

6.8. Otimização do Tempo de Arranque

O tempo entre a ligação do dispositivo à porta USB e a disponibilidade efetiva do menu de diagnóstico foi identificado, desde o início do trabalho, como um fator relevante de usabilidade em contexto industrial, onde cada segundo de espera tem um custo percebido pelo operador. Esta secção descreve o processo de otimização implementado para reduzir a dependência de serviços não essenciais ao funcionamento do dispositivo.

O arranque original do protótipo tinha o dispositivo composto a ser configurado tardiamente através de *rc.local*, um mecanismo de arranque legado que só corre depois de o sistema atingir o alvo *multi-user.target*, ou seja, depois de todos os serviços de sistema terem terminado a sua inicialização, incluindo os desnecessários ao funcionamento do dispositivo. Uma primeira ronda de otimizações, incluindo a remoção de esperas fixas (*sleep*) no arranque e a desativação de serviços desnecessários, permitiu melhorar este comportamento, sem alterar a arquitetura do arranque.

A mudança mais significativa consistiu em substituir o arranque via *rc.local* por três unidades *systemd* próprias: *usb-gadget.service* (configuração do *gadget* USB composto), *oled-splash.service* (ecrã de arranque no OLED) e *diag-tool.service* (aplicação de menu). Esta reorganização permitiu separar responsabilidades e controlar melhor a ordem de arranque dos componentes necessários ao funcionamento do protótipo. Essa análise revelou que o menu dependia, por omissão, de *multi-user.target*, que só era atingido depois do serviço *dhcpcd* terminar a negociação de rede, necessária apenas para o acesso SSH de desenvolvimento pela interface ECM (secção 6.3), não para o funcionamento do dispositivo em si.

Ao reconfigurar *diag-tool.service* para depender diretamente de *usb-gadget.service* em vez de *multi-user.target*, o menu deixou de esperar pela negociação de rede, avançando logo que o *gadget* USB estivesse pronto. Complementarmente, o ecrã de arranque (*oled-splash.service*), iniciado o mais cedo possível a seguir a *sysinit.target*, apresenta a mensagem “A carregar...” no ecrã OLED enquanto o *gadget* USB ainda está a ser configurado, dando ao operador feedback visual imediato em vez de um ecrã apagado; como o controlador SSD1306 mantém a última imagem escrita mesmo sem comunicação adicional, a transição para o menu ocorre sem necessidade de qualquer sinalização explícita entre os dois processos.

Foi ainda avaliada uma via mais radical, reconstruir o arranque em torno de uma *initramfs* dedicada, que permitiria em teoria acender o ecrã OLED nos primeiros segundos do arranque, muito antes do subsistema I²C estar disponível pela via normal do *systemd*. Esta via foi conscientemente posta de lado: implica reconstruir uma parte sensível do processo de arranque do único protótipo físico disponível, com risco real de deixar o cartão microSD sem arrancar, e o tempo disponível até à entrega final não permitia recuperar desse risco com segurança. A opção adotada representa, assim, um compromisso deliberado entre o ganho de usabilidade e o risco de engenharia, ficando a via da *initramfs* registada como trabalho futuro no capítulo 8.

Por último, os serviços de rede (*dhcpcd*, SSH) foram deliberadamente mantidos ativos nesta fase do projeto, por serem necessários ao acesso de desenvolvimento pela interface ECM; num dispositivo preparado para produção, sem necessidade de acesso remoto contínuo, a sua desativação poderá reduzir o tempo de arranque total do

sistema operativo, embora o menu de diagnóstico já não dependa diretamente dessa cadeia de serviços.

7. Testes e Validação

Este capítulo apresenta os testes realizados ao protótipo desenvolvido, com o objetivo de validar o funcionamento integrado da solução. A validação incidiu sobre a interação com a HMI, a seleção e execução das rotinas de diagnóstico, a verificação da assinatura digital dos *scripts*, a injeção automática de comandos através da interface HID, a escrita dos resultados na unidade *Mass Storage* e a posterior leitura desses resultados pelo dispositivo. Foi também avaliado o tempo de arranque do protótipo, por se tratar de um fator relevante para a utilização em contexto de manutenção. Por fim, são identificadas as principais limitações da validação efetuada.

7.1. Ambiente de Teste

Todos os testes descritos neste capítulo foram realizados numa única máquina anfitriã, com as seguintes características: computador do tipo PC, equipado com um processador AMD Ryzen 7 5700U com gráfica Radeon integrada (16 cores), 16 GB de memória RAM DDR4, porta USB-A 2.0 e sistema operativo Ubuntu 24.04.4 LTS.

O protótipo utilizado nos testes foi o dispositivo descrito no capítulo 6, baseado no Raspberry Pi Zero WH e ligado diretamente à máquina anfitriã através da porta USB configurada em modo dispositivo. Esta ligação permitiu disponibilizar, durante os testes, as interfaces HID, *Mass Storage* e ECM, conforme descrito na secção 6.3. A interação com o protótipo foi realizada através da HMI física, composta pelo ecrã OLED SSD1306 e pelos três botões de navegação.

Esta é uma limitação deliberadamente assumida e não uma omissão: o planeamento do projeto não previu o acesso a um grande leque de máquinas com sistemas operativos diferentes, pelo que os resultados apresentados devem ser lidos como validação da correção do mecanismo nesta configuração concreta, e não como garantia de comportamento idêntico em qualquer combinação possível de sistema operativo e disposição de teclado. O mecanismo de deteção automática da disposição de teclado (secção 6.7) foi concebido precisamente para mitigar esta limitação em uso real, mas foi ele próprio apenas validado, por construção, contra a disposição disponível nesta máquina de teste.

7.2. Metodologia de Validação

Cada um dos cinco *scripts* sem privilégios de administrador, descritos na secção 6.5.1, foram executados repetidamente através do menu real do dispositivo, não por invocação direta na máquina anfitriã, de forma a validar a cadeia completa: leitura da disposição de teclado configurada, verificação da assinatura digital, injeção do comando via HID, execução na máquina anfitriã, escrita do resultado na unidade *Mass Storage* e leitura do resultado pela Raspberry Pi através de *mttools*.

Para cada rotina de diagnóstico, o operador selecionou o *script* pretendido através da HMI, confirmou a execução e aguardou pela indicação de estado no ecrã OLED. Este procedimento permitiu validar não apenas os *scripts* isoladamente, mas

também a integração entre a interface física, o *firmware* da aplicação, a verificação de integridade, a interface HID e a disponibilização dos resultados.

A validação considerou também situações de erro controladas, nomeadamente a alteração deliberada de um *script* já assinado, sem regenerar a respetiva assinatura. Este teste teve como objetivo confirmar que o mecanismo de segurança aborta a execução antes de qualquer comando ser injetado na máquina anfitriã.

As medições do tempo de arranque foram realizadas separadamente, recorrendo à ferramenta *systemd-analyze* e à análise da cadeia crítica de dependências do *systemd*. Estas medições permitiram comparar a configuração inicial, a configuração com otimizações pontuais e a configuração final baseada em unidades *systemd* dedicadas. Os resultados correspondentes são apresentados na secção 7.3.

7.3. Resultados dos Testes Funcionais

Os testes funcionais incidiram sobre os principais mecanismos do protótipo: execução dos *scripts* de diagnóstico, verificação da assinatura digital, injeção de comandos por HID, escrita dos resultados na unidade *Mass Storage*, leitura dos resultados pelo dispositivo e apresentação do estado da execução no ecrã OLED.

Em todas as nove execuções registadas, o ficheiro de resultado recuperado pela Raspberry Pi terminava com **DIAG_EXIT=0**, confirmado pelo ecrã OLED com SUCESSO. Este comportamento indica que os *scripts* foram executados corretamente na máquina anfitriã, que os resultados foram escritos na unidade *Mass Storage* e que o dispositivo conseguiu recuperá-los através de *mttools*. A título de exemplo de resultado obtido, reproduz-se abaixo o resultado de uma execução do diagnóstico de *uptime*:

```
DIAGNOSTICO DE UPTIME
Data: 2026-07-04 17:20:08
Host: HostDeTeste

Uptime           : 0d 1h 37min

Arranque          : 2026-07-04 15:43:06

Load average     : 2.68 (1min) 1.55 (5min) 1.11 (15min)

FIM
DIAG_EXIT=0
```

O resultado apresentado segue o formato definido para os *scripts* de diagnóstico, incluindo a identificação do diagnóstico executado, a data de execução, a máquina anfitriã, os dados recolhidos e o código final **DIAG_EXIT**. A presença de **DIAG_EXIT=0** permite distinguir uma execução concluída com sucesso de uma falha do próprio *script* ou de uma falha na recolha dos resultados.

O mecanismo de deteção de adulteração, descrito na secção 6.6, foi também testado deliberadamente: alterando um único byte de um *script* já assinado na área onde se encontram os *scripts*, sem regenerar a respetiva assinatura, a tentativa de execução seguinte foi corretamente rejeitada antes de qualquer tecla ser injetada na máquina anfitriã, com o ecrã a apresentar **ADULTERADO**, confirmando o comportamento descrito na secção 6.6.

Tabela 6: Síntese dos testes funcionais realizados

Teste	Objetivo	Resultado obtido
Execução dos <i>scripts</i> sem privilégios	Confirmar a execução das rotinas de diagnóstico através do menu do dispositivo	Execução concluída com DIAG_EXIT=0
Escrita dos resultados	Confirmar que a máquina anfitriã escreve o ficheiro de resultado na unidade <i>Mass Storage</i>	Ficheiros dos resultados criados corretamente
Leitura dos resultados	Confirmar que o dispositivo recupera os resultados através de mtools	Resultados lidos e estado apresentado no ecrã OLED
Assinatura válida	Confirmar que <i>scripts</i> assinados são aceites para execução	Execução permitida
<i>Script</i> adulterado	Confirmar que alterações não autorizadas são detetadas	Execução rejeitada com estado ADULTERADO
Compatibilidade de teclado	Confirmar que os comandos injetados correspondem à disposição configurada	Comandos executados sem erros de caracteres nos testes realizados

Em conjunto, estes resultados confirmam que a cadeia completa do dispositivo, incluindo a deteção da disposição de teclado, a verificação de integridade, a injeção HID e a recolha de resultados por *Mass Storage*, funciona de forma consistente na configuração testada.

O tempo de arranque foi testado por ser um fator relevante para a usabilidade do protótipo. Neste contexto, foi distinguido o tempo total de arranque do sistema operativo do tempo até à disponibilidade efetiva do menu de diagnóstico, uma vez que é este último que determina quando o operador pode começar a utilizar o dispositivo.

As medições foram realizadas com a ferramenta `systemd-analyze`^[15], nativa do sistema operativo, e com a análise da cadeia crítica de dependências através de `systemd-analyze critical-chain`. Foram comparadas três fases do desenvolvimento: a configuração inicial baseada em `rc.local`, uma configuração intermédia com otimizações pontuais e a configuração final baseada em unidades `systemd` dedicadas, descrita na secção 6.8.

Na configuração inicial, o arranque situava-se em torno dos 55 segundos, com o dispositivo composto a ser configurado tardiamente através de `rc.local`. Após uma primeira ronda de otimizações, incluindo a remoção de esperas fixas (*sleep*) e a desativação de serviços desnecessários, o tempo até à disponibilidade do menu foi reduzido para cerca de 43 segundos, ainda sem alterar a arquitetura geral do arranque.

A configuração final passou a usar as unidades *usb-gadget.service*, *oled-splash.service* e *diag-tool.service*. Nesta versão, o menu deixou de depender de *multi-user.target* e passou a depender diretamente de *usb-gadget.service*, permitindo que a aplicação de menu fosse iniciada logo após a configuração do dispositivo USB composto.

A Figura 9 documenta a medição final: o arranque completo do sistema operativo, incluindo os serviços de rede mantidos ativos para acesso de desenvolvimento pela interface ECM, demora cerca de 42 segundos, mas o menu de diagnóstico (*diag-tool.service*) fica disponível entre os 21 e os 23 segundos, por já não depender dessa cadeia. A cadeia crítica mostra concretamente a origem deste valor: *usb-gadget.service* fica ativo aos 19,3 segundos, demorando 2,6 segundos a arrancar, e *diag-tool.service* arranca 1,8 segundos depois disso.

```

raspberrypi@admin:~ $ systemd-analyze
Startup finished in 4.067s (kernel) + 37.998s (userspace) = 42.066s
multi-user.target reached after 37.691s in userspace
raspberrypi@admin:~ $ systemd-analyze critical-chain diag-tool.service
The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

diag-tool.service +1.788s
└─usb-gadget.service @19.311s +2.642s
   └─basic.target @17.743s
      └─sockets.target @17.733s
         └─triggerhappy.socket @17.714s
            └─sysinit.target @17.544s
               └─systemd-update-utmp.service @16.984s +540ms
                  └─systemd-tmpfiles-setup.service @16.161s +771ms
                     └─local-fs.target @15.884s
                        └─boot.mount @15.733s +138ms
                           └─systemd-fsck@dev-disk-by\x2dpartuuid-e6df4875\x2d01.service>
                              └─dev-disk-by\x2dpartuuid-e6df4875\x2d01.device @14.798s

```

Figura 9 - Medição final do tempo de arranque

A Tabela 7 sintetiza a evolução do tempo até à disponibilidade do menu de diagnóstico ao longo das configurações testadas.

Tabela 7: Evolução do tempo até à disponibilidade do menu de diagnóstico

Fase	Configuração	Tempo até ao menu operacional
Inicial (relatório intermédio)	<i>rc.local</i> , dependente de <i>multi-user.target</i>	≈ 55 s
Otimizações pontuais	<i>rc.local</i> , sem esperas fixas, menos serviços	≈ 43 s
Final	Unidades <i>systemd</i> dedicadas, desacopladas de <i>multi-user.target</i>	≈ 21-23 s

Os resultados obtidos mostram que a reorganização do arranque permitiu reduzir o tempo de espera do operador para menos de metade do valor inicial. Embora o sistema operativo continue a concluir o arranque completo apenas por volta dos 42

segundos, o menu de diagnóstico fica disponível significativamente mais cedo, o que melhora a utilização prática do dispositivo em contexto de manutenção.

7.4. Limitações da Validação

Apesar de os testes realizados confirmarem o funcionamento integrado do protótipo na configuração utilizada, a validação efetuada apresenta algumas limitações. Em primeiro lugar, os testes foram realizados numa única máquina anfitriã, com Ubuntu 24.04.4 LTS e disposição de teclado em português. Assim, os resultados obtidos validam o comportamento do protótipo nesta configuração, mas não garantem o mesmo funcionamento em outros sistemas operativos, distribuições Linux, versões de terminal ou disposições de teclado.

A validação foi também realizada em ambiente controlado de laboratório, e não em contexto industrial real. Não foram, por isso, avaliadas condições como utilização prolongada, diferentes perfis de operador, políticas de segurança mais restritivas, portas USB bloqueadas ou configurações específicas de permissões. Do mesmo modo, a compatibilidade de teclado foi testada apenas com a disposição disponível na máquina anfitriã, apesar de o mecanismo descrito na secção 6.7 ter sido concebido para suportar outras disposições.

Relativamente aos *scripts* com privilégios de administrador, a validação deve ser interpretada com cautela, uma vez que estes dependem da credencial correta estar registada no cofre local de credenciais. Também o mecanismo de assinatura digital foi validado apenas no cenário de adulteração considerado, não protegendo contra acesso físico direto e prolongado ao próprio dispositivo, como referido na secção 6.6.

Estas limitações não invalidam os resultados obtidos, mas delimitam o seu alcance: o protótipo demonstrou funcionar corretamente na configuração testada, ficando por realizar uma validação mais abrangente em diferentes sistemas, disposições de teclado e cenários reais de manutenção.

8. Conclusão e Trabalho Futuro

Este projeto teve como objetivo o desenvolvimento de um dispositivo portátil capaz de automatizar a execução de diagnósticos em ambiente industrial, sem exigir formação avançada do operador nem depender da introdução manual de comandos numa interface de linha de comandos. A solução procurou, assim, simplificar tarefas de manutenção de primeira linha, permitindo que rotinas previamente definidas e validadas fossem executadas de forma automática, controlada e com menor probabilidade de erro operacional.

O protótipo desenvolvido concretiza este objetivo através de um dispositivo baseado em Raspberry Pi Zero WH, com uma interface física composta por ecrã OLED e botões de navegação. Do ponto de vista da máquina anfitriã, o dispositivo é reconhecido como um periférico USB composto, disponibilizando uma interface HID para a injeção automática de comandos e uma interface *Mass Storage* para guardar e disponibilizar os resultados obtidos. Foi ainda mantida uma interface ECM como apoio ao desenvolvimento e depuração. A solução integra cinco scripts de diagnóstico sem privilégios de administrador, suporte a diferentes disposições de teclado, verificação de integridade dos scripts por assinatura digital e otimizações no processo de arranque.

Todas estas extensões foram realizadas em software, sem necessidade de componentes adicionais de hardware, mantendo o custo estimado do protótipo abaixo dos 35 €.

Os testes realizados permitiram validar o funcionamento integrado da cadeia principal do sistema. Através da HMI, o operador consegue selecionar uma rotina de diagnóstico, o dispositivo verifica a assinatura digital do *script* correspondente, injeta os comandos necessários através da interface HID, a máquina anfitriã executa o diagnóstico e os resultados são escritos na unidade *Mass Storage*, sendo posteriormente lidos pelo próprio dispositivo. Em todas as execuções registadas dos *scripts* sem privilégios, os resultados foram recuperados com sucesso e o estado final foi apresentado no ecrã OLED. Foi também confirmada a deteção de *scripts* adulterados e a redução do tempo até à disponibilidade do menu de diagnóstico para cerca de 21 a 23 segundos. Assim, face aos objetivos definidos, considera-se que o protótipo demonstra a viabilidade da solução proposta. No entanto, esta validação foi realizada numa única máquina anfitriã e em ambiente controlado, pelo que os resultados devem ser interpretados como validação do protótipo nessa configuração concreta, e não como garantia de funcionamento universal em todos os sistemas operativos, disposições de teclado ou contextos industriais.

Como trabalho futuro, identificam-se as seguintes direções:

- **Validação alargada:** repetir os testes num conjunto mais amplo de sistemas operativos, distribuições Linux, disposições de teclado e máquinas anfitriãs, para além da máquina única usada neste relatório.
- **Consolidação dos *scripts* com privilégios de administrador:** rever e validar os *scripts* que dependem de permissões elevadas, nomeadamente os relativos a *logs* do sistema e ficheiros protegidos, testando o mecanismo de credenciais e injeção de password em mais máquinas anfitriãs.
- **Configuração de produção:** desativar os serviços de rede mantidos ativos para desenvolvimento, nomeadamente os associados à interface ECM e ao acesso SSH, numa imagem destinada a instalação final, reduzindo o tempo de arranque total do sistema operativo e diminuindo a superfície de exposição do dispositivo.
- **Arranque por *initramfs*:** explorar a abordagem de maior risco, que permitiria antecipar o ecrã de arranque e a interface visual para os instantes iniciais do processo de arranque do sistema.
- **Evolução física e funcional do protótipo:** desenvolver uma caixa física adequada à utilização em campo, melhorar a robustez da HMI e expandir a biblioteca de diagnósticos disponíveis.

Em suma, o protótipo desenvolvido demonstra, em hardware real e face a um enquadramento de mercado sem alternativas comparáveis, que é possível reunir num único dispositivo de baixo custo a emulação simultânea de teclado e armazenamento, a seleção de diagnósticos em tempo de execução, a verificação de integridade dos *scripts* executados e a compatibilidade com múltiplas disposições de teclado. Deste modo, o trabalho desenvolvido cumpre o objetivo central do projeto: tornar mais acessível e controlada a automação de diagnósticos técnicos em contexto de manutenção industrial.

Referências

- [1] C. Peacock, "USB in a NutShell – Making sense of the USB standard," *Beyond Logic*. [Online]. Disponível em: <https://www.beyondlogic.org/usbnutshell/usb1.shtml> [Acedido em: 15/04/2026].
- [2] C. Peacock, "USB in a NutShell – Chapter 8: USB Classes (HID)," *Beyond Logic*. [Online]. Disponível em: <https://www.beyondlogic.org/usbnutshell/usb8.shtml> [Acedido em: 15/04/2026].
- [3] C. Peacock, "USB in a NutShell – Chapter 10: USB Mass Storage," *Beyond Logic*. [Online]. Disponível em: <https://www.beyondlogic.org/usbnutshell/usb10.shtml> [Acedido em: 15/04/2026].
- [4] Linux Kernel, "USB Gadget Configfs Documentation." [Online]. Disponível em: https://docs.kernel.org/usb/gadget_configfs.html [Acedido em: 20/04/2026].
- [5] Random Nerd Tutorials, "Raspberry Pi Pico: SSD1306 OLED Display (MicroPython)." [Online]. Disponível em: <https://randomnerdtutorials.com/raspberry-pi-pico-ssd1306-oled-micropython/> [Acedido em: 01/03/2026].
- [6] SparkFun Electronics, "I2C," *SparkFun Learn*. [Online]. Disponível em: <https://learn.sparkfun.com/tutorials/i2c>. [Acedido em: 01/03/2026].
- [7] Raspberry Pi Foundation, "Raspberry Pi Zero WH Product Brief." [Online]. Disponível em: <https://www.raspberrypi.com/products/raspberry-pi-zero-w/> [Acedido em: 27/02/2026].
- [8] joan2937, "pigpio – C library for the Raspberry Pi which allows control of the GPIO." [Online]. Disponível em: <https://abyz.me.uk/rpi/pigpio/> [Acedido em: 05/03/2026].
- [9] Hak5, "USB Rubber Ducky – Product Page." [Online]. Disponível em: <https://shop.hak5.org/products/usb-rubber-ducky> [Acedido em: 22/02/2026].
- [10] Hak5, "Bash Bunny – Product Page." [Online]. Disponível em: <https://shop.hak5.org/products/bash-bunny> [Acedido em: 24/02/2026].
- [11] Flipper Devices, "Flipper Zero – Product Page." [Online]. Disponível em: <https://flipper.net/products/flipper-zero> [Acedido em: 25/02/2026].
- [12] Flipper Devices, "Flipper Zero Documentation – Bad USB." [Online]. Disponível em: <https://docs.flipper.net/zero/bad-usb> [Acedido em: 25/02/2026].
- [13] Cloudflare, "What Is the OSI Model?," *Cloudflare Learning Center*. [Online]. Disponível em: <https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/> [Acedido em: 18/04/2026].
- [14] OpenSSL Project, "openssl-dgst." [Online]. Disponível em: <https://www.openssl.org/docs/manmaster/man1/openssl-dgst.html> [Acedido em: 12/06/2026].
- [15] The systemd Project, "systemd-analyze." [Online]. Disponível em: <https://www.freedesktop.org/software/systemd/man/latest/systemd-analyze.html> [Acedido em: 10/06/2026].
- [16] GNU Project, "Mtools User's Guide." [Online]. Disponível em: <https://www.gnu.org/software/mtools/manual/mtools.html> [Acedido em: 19/04/2026].
- [17] USB Implementers Forum, "HID Usage Tables, Version 1.12." [Online]. Disponível em: https://www.usb.org/sites/default/files/documents/hut1_12v2.pdf [Acedido em: 22/05/2026].
- [18] D. Gamble, "cJSON – Ultralightweight JSON Parser in C." [Online]. Disponível em: <https://github.com/DaveGamble/cJSON> [Acedido em: 25/05/2026].